CHAPTER 1

Introduction to language models

1.1 The sequence prediction problem and probabilistic language modeling

At its core, a language model formalizes the intuitive mechanism by which humans evaluate the fluency, plausibility, and likelihood of word sequences. In Natural Language Processing (NLP) and modern AI-assisted Software Engineering, we model human language and source code not through rigid, brittle rule systems, but as a discrete stochastic process governed by probability distributions.

Let $\mathcal{V} = \{w_1, w_2, \ldots, w_V\}$ denote a finite vocabulary of $V = |\mathcal{V}|$ discrete tokens (e.g., words, subwords, or punctuation symbols). An arbitrary natural language sentence, code snippet, or document of length N is represented as an ordered sequence of discrete random variables $W = (w_1, w_2, \ldots, w_N) \in \mathcal{V}^N$.

Definition 1.1: *Statistical language model*

A *statistical language model* is a formal probability distribution $\mathbb{P}$ defined over the set of all possible discrete sequences $\mathcal{S} = \bigcup_{N=1}^{\infty} \mathcal{V}^N$. For any specific sequence $W = (w_1, \ldots, w_N)$, the language model assigns a scalar probability $\mathbb{P}(W) \in [0, 1]$ satisfying the Kolmogorov probability axioms:

$$\mathbb{P}(W) \geq 0 \quad \forall W \in \mathcal{S}, \quad \text{and} \quad \sum_{S \in \mathcal{S}} \mathbb{P}(S) = 1. \tag{1.1}$$

The model maps the syntactic and semantic conventions of human language into real values on the unit interval:

- Fluent, grammatically coherent sentences receive high probability mass:

$$\mathbb{P}(\text{“this”}, \text{“is”}, \text{“a”}, \text{“reasonable”}, \text{“sentence”}) = 0.1$$

- Syntactically valid but semantically unusual or rare sentences receive noticeably lower probability:

$$\mathbb{P}(\text{“this”}, \text{“is”}, \text{“a”}, \text{“purple”}, \text{“sentence”}) = 0.01$$

- Ungrammatical, corrupted, or disjoint sequences receive negligible probability:

$$\mathbb{P}(\text{“this”}, \text{“are”}, \text{“a”}, \text{“reasonable”}, \text{“sentence”}) = 0.001$$

Evaluating the joint probability $\mathbb{P}(w_1, \ldots, w_N)$ directly over the combinatorial space $\mathcal{V}^N$ is intractable because observing every conceivable multi-word sequence in a training dataset

is impossible. To structure this estimation problem, we invoke the fundamental *chain rule of probability*.

Definition 1.2: *Chain rule decomposition of language models*

For any sequence of random variables $(w_1, w_2, \ldots, w_N)$, the joint probability factors identically and unconditionally into an ordered product of conditional next-token probabilities:

$$\begin{aligned}\mathbb{P}(w_1, w_2, \ldots, w_N) &= \mathbb{P}(w_1) \cdot \mathbb{P}(w_2 \mid w_1) \cdot \mathbb{P}(w_3 \mid w_1, w_2) \cdots \mathbb{P}(w_N \mid w_1, \ldots, w_{N-1}) \\ &= \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_1, \ldots, w_{t-1}) = \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_{<t}),\end{aligned} \tag{1.2}$$

where $w_{<t} = (w_1, \ldots, w_{t-1})$ represents the prefix history preceding timestep t. For the initial token $t = 1$, the conditioning prefix is the empty sequence $\emptyset$, giving $\mathbb{P}(w_1 \mid w_{<1}) = \mathbb{P}(w_1)$.

Applying Equation (1.2) to our running example yields the concrete factorization:

$$\begin{aligned}\mathbb{P}(\text{this, is, a, reasonable, sentence}) = \mathbb{P}(&\text{this}) \\ &\cdot \mathbb{P}(\text{is} \mid \text{this}) \\ &\cdot \mathbb{P}(\text{a} \mid \text{this, is}) \\ &\cdot \mathbb{P}(\text{reasonable} \mid \text{this, is, a}) \\ &\cdot \mathbb{P}(\text{sentence} \mid \text{this, is, a, reasonable}).\end{aligned}$$

Reframing language modeling as conditional prediction. This mathematical decomposition marks a pivotal conceptual shift in Natural Language Processing. Instead of attempting to score complete, static documents simultaneously, the modeling focus shifts entirely to sequential, conditional next-word estimation:

Given history $w_{<t}$, estimate the categorical probability distribution over all words in $\mathcal{V}$: $\mathbb{P}(w_t = w \mid w_{<t})$.

This formulation aligns directly with the classic psychological *cloze test* (fill-in-the-blank):

"This is a reasonable _____*"* $\implies$ $\mathbb{P}(\text{sentence} \mid \text{This is a reasonable}) = ?$

Every modern autoregressive language model, including GPT-4 and Claude, relies fundamentally on this conditional next-token formulation.

Intuition & analogy: The core thesis: next-word prediction as full language modeling

In the opening lecture of the Politecnico di Torino course, the fundamental essence of modern language modeling was crystallized:

> *"If we can predict the next word, then we have estimated this conditional probability. And then we can plug this into the chain rule, and estimate the probability of any arbitrary sentence. This is what a language model does: nothing more, nothing less. If you understand this idea, you understand the core mechanism driving the entire course, from 1980s Markov chains to modern trillion-parameter LLMs."*

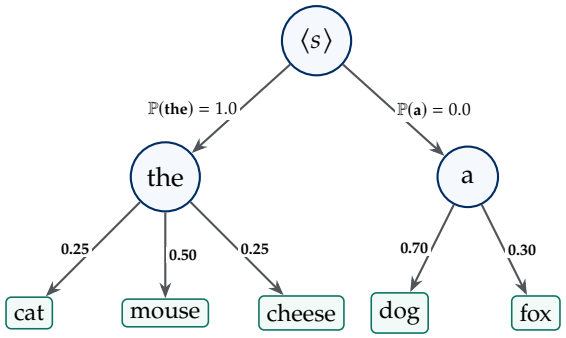


Figure 1.1: Visualizing autoregressive sequence generation as path traversal across a probability branching tree. At the root symbol $\langle s \rangle$, outgoing edges partition total probability mass across vocabulary tokens. At each node $w_{<t}$, children represent the conditional distribution $\mathbb{P}(w_t \mid w_{<t})$. The law of total probability guarantees that branch probabilities sum to unity at each decision point.

Intuition & analogy: The branching probability tree

Sequence generation can be visualized geometrically as walking down a directed decision tree (Figure 1.1). At the root node $\langle s \rangle$, the model confronts the entire vocabulary $\mathcal{V}$. Choosing a token w_1 traverses an edge with probability $\mathbb{P}(w_1)$ and lands at a new conditioning state. From this state, another set of branches opens, weighted by $\mathbb{P}(w_2 \mid w_1)$. The joint probability of an entire sentence is simply the product of edge weights along that unique trajectory. Generating text consists of sampling a path from the root to a designated terminal token $\langle /s \rangle$.

1.2 Markov assumptions and n-gram language models

Although Equation (1.2) is mathematically exact and preserves all syntactic and semantic dependencies, computing $\mathbb{P}(w_t \mid w_{<t})$ directly in empirical applications presents an insurmountable combinatorial bottleneck. As the sequence index t grows, the conditioning history $w_{<t} = (w_1, \ldots, w_{t-1})$ expands monotonically. Because each constituent token is drawn from a vocabulary $\mathcal{V}$ of size $V = |\mathcal{V}|$, the space of potential prefix histories preceding token t is the Cartesian product space $\mathcal{V}^{t-1}$, whose cardinality is:

$$|\mathcal{H}_t| = |\mathcal{V}|^{t-1} = V^{t-1}. \tag{1.3}$$

Even for a modest vocabulary of $V = 10{,}000$ words, tracking histories of length $t = 4$ requires estimating probabilities over $10{,}000^3 = 10^{12}$ distinct conditioning states. In any finite training corpus, almost every sequence of length greater than three occurs exactly zero times.

To make estimation mathematically tractable, the principle of conditional independence formalizes that future tokens depend only on a bounded historical window. In the landmark 1948 treatise on information theory [**?**], it was demonstrated that modeling natural language as a discrete Markov process produces increasingly coherent synthetic English as the conditioning order increases. Modern pedagogical treatments [**?**] formalize this simplification as the foundation of classical statistical natural language processing.

Definition 1.3: *Markov assumption of order $n-1$*

An n-gram language model assumes that the conditional probability of the current

word w_t depends exclusively on the immediately preceding $n-1$ words, rendering it conditionally independent of all earlier discourse history:

$$\mathbb{P}(w_t \mid w_1, w_2, \ldots, w_{t-1}) \approx \mathbb{P}(w_t \mid w_{t-n+1}, \ldots, w_{t-1}). \tag{1.4}$$

The integer $n \geq 1$ dictates the order of the model:

- **Unigram Model ($n = 1$, Order 0)**: Completely memoryless. Each word is treated as statistically independent of all past and future tokens:

$$\mathbb{P}(w_1, \ldots, w_N) \approx \prod_{t=1}^{N} \mathbb{P}(w_t).$$

- **Bigram Model ($n = 2$, Order 1)**: First-order Markov chain. Conditioning depends exclusively on the single preceding token w_{t-1}:

$$\mathbb{P}(w_1, \ldots, w_N) \approx \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_{t-1}).$$

- **Trigram Model ($n = 3$, Order 2)**: Second-order Markov chain. Conditioning depends on the preceding pair of tokens (w_{t-2}, w_{t-1}):

$$\mathbb{P}(w_1, \ldots, w_N) \approx \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_{t-2}, w_{t-1}).$$

1.2.1 Combinatorial state-space reduction

The primary mathematical virtue of the Markov assumption is that it truncates an exponentially growing state space into a fixed, stationary cardinality. Under an n-gram model, the conditioning context is always restricted to $n-1$ words, regardless of the absolute sequence position t:

$$|\mathcal{H}^{(n)}| = |\mathcal{V}|^{n-1} = V^{n-1}. \tag{1.5}$$

Consequently, specifying the complete transition probability distribution across the entire vocabulary requires an $(n-1)$-dimensional conditioning tensor containing:

$$P_{\text{params}} = V^{n-1} \times (V - 1) \approx V^n \tag{1.6}$$

free parameters. Table 1.1 contrasts the unconstrained historical state space with fixed n-gram models for realistic vocabulary sizes.

While the unconstrained state space exceeds the total number of atoms in the observable universe ($\approx 10^{80}$) by step $t = 20$, a bigram model maintains a manageable state space of 50,000 rows. However, this drastic dimensional reduction comes at the cost of complete amnesia for any context preceding the immediate $n-1$ tokens.

1.2.2 Maximum likelihood estimation (MLE)

Under Maximum Likelihood Estimation, we estimate the conditional transition probabilities directly by counting occurrences of exact token sequences in a training corpus [**?**]. For a generic n-gram:

$$\mathbb{P}_{\text{MLE}}(w_t \mid w_{t-n+1}, \ldots, w_{t-1}) = \frac{C(w_{t-n+1}, \ldots, w_{t-1}, w_t)}{C(w_{t-n+1}, \ldots, w_{t-1})}, \tag{1.7}$$

Table 1.1: State-space cardinality $|\mathcal{H}|$ as a function of sequence position t versus fixed n-gram Markov orders for vocabularies $V = 10{,}000$ and $V = 50{,}000$.

Model / Order	**Effective Context**	**Formula**	**States ($V = 10^4$)**	**States ($V = 5 \times 10^4$)**
Unconstrained ($t = 2$)	1 previous token	V^1	1.0×10^4	5.0×10^4
Unconstrained ($t = 3$)	2 previous tokens	V^2	1.0×10^8	2.5×10^9
Unconstrained ($t = 5$)	4 previous tokens	V^4	1.0×10^{16}	6.25×10^{18}
Unconstrained ($t = 10$)	9 previous tokens	V^9	1.0×10^{36}	1.95×10^{42}
Unconstrained ($t = 20$)	19 previous tokens	V^{19}	1.0×10^{76}	1.90×10^{89}
Unigram ($n = 1$)	0 tokens (memoryless)	$V^0 = 1$	1	1
Bigram ($n = 2$)	1 previous token	V^1	1.0×10^4	5.0×10^4
Trigram ($n = 3$)	2 previous tokens	V^2	1.0×10^8	2.5×10^9
4-gram ($n = 4$)	3 previous tokens	V^3	1.0×10^{12}	1.25×10^{14}

where $C(\cdot)$ denotes the raw frequency of that exact sequence in the training text. For a bigram model ($n = 2$), Equation (1.7) specializes to:

$$\mathbb{P}_{\text{MLE}}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t)}{C(w_{t-1})} = \frac{C(w_{t-1}, w_t)}{\sum_{w \in \mathcal{V}} C(w_{t-1}, w)}. \tag{1.8}$$

Normalizing each row by the unigram count of the conditioning prefix ensures that $\sum_{w \in \mathcal{V}} \mathbb{P}_{\text{MLE}}(w \mid w_{t-1}) = 1$.

1.3 Transition count matrix construction walkthrough

To understand exactly how statistical n-gram language models operate, we trace the step-by-step extraction of bigram frequencies and transition probabilities from the authoritative lecture corpus:

1. Sentence 1: *"The cat chased the mouse happily"*
2. Sentence 2: *"The mouse ate the cheese"*

Vocabulary extraction. Extracting all unique tokens (case-normalized with standard initial capitalization for clarity) yields a vocabulary of size $|V| = 7$:

$$\mathcal{V} = \{\text{Ate, Cat, Chased, Cheese, Happily, Mouse, The}\}.$$

Sequential bigram decomposition. We iterate through both training sentences token by token, recording each observed transition $(w_{t-1} \to w_t)$:

- From Sentence 1 (*"The cat chased the mouse happily"*):
  1. $(\text{The}, \text{cat}) \implies C(\text{The}, \text{Cat}) \leftarrow 1$
  2. $(\text{cat}, \text{chased}) \implies C(\text{Cat}, \text{Chased}) \leftarrow 1$
  3. $(\text{chased}, \text{the}) \implies C(\text{Chased}, \text{The}) \leftarrow 1$
  4. $(\text{the}, \text{mouse}) \implies C(\text{The}, \text{Mouse}) \leftarrow 1$
  5. $(\text{mouse}, \text{happily}) \implies C(\text{Mouse}, \text{Happily}) \leftarrow 1$
- From Sentence 2 (*"The mouse ate the cheese"*):

6. $(\text{The}, \text{mouse}) \implies C(\text{The}, \text{Mouse}) \leftarrow 2$
7. $(\text{mouse}, \text{ate}) \implies C(\text{Mouse}, \text{Ate}) \leftarrow 1$
8. $(\text{ate}, \text{the}) \implies C(\text{Ate}, \text{The}) \leftarrow 1$
9. $(\text{the}, \text{cheese}) \implies C(\text{The}, \text{Cheese}) \leftarrow 1$

Unigram normalization counts. Summing across rows provides the total unigram occurrences $C(w_{t-1})$:

$$\begin{aligned} C(\text{The}) &= 4 \quad (\text{cat} : 1, \text{mouse} : 2, \text{cheese} : 1) \\ C(\text{Cat}) &= 1 \quad (\text{chased} : 1) \\ C(\text{Chased}) &= 1 \quad (\text{the} : 1) \\ C(\text{Mouse}) &= 2 \quad (\text{happily} : 1, \text{ate} : 1) \\ C(\text{Ate}) &= 1 \quad (\text{the} : 1) \\ C(\text{Cheese}) &= 0 \quad (\text{terminal word, no outgoing transitions}) \\ C(\text{Happily}) &= 0 \quad (\text{terminal word, no outgoing transitions}) \end{aligned}$$

Full empirical transition probability matrix. Applying Equation (1.8) yields the complete 7×7 transition matrix $\mathbf{M} \in \mathbb{R}^{7\times7}$ where row i represents condition w_{t-1} and column j represents next token w_t. Table 1.2 presents both raw counts and normalized MLE probabilities.

Table 1.2: Empirical transition count matrix and normalized bigram conditional probabilities $\mathbb{P}(w_t \mid w_{t-1})$ for vocabulary $\mathcal{V} = \{\text{Ate, Cat, Chased, Cheese, Happily, Mouse, The}\}$.

Condition (w_{t-1})	Ate	Cat	Chased	Cheese	Happily	Mouse	The	Row Total
Ate	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	1 (1.00)	1
Cat	0 (0.0)	0 (0.0)	1 (1.00)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	1
Chased	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	1 (1.00)	1
Cheese	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0
Happily	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0
Mouse	1 (0.50)	0 (0.0)	0 (0.0)	0 (0.0)	1 (0.50)	0 (0.0)	0 (0.0)	2
The	0 (0.0)	1 (0.25)	0 (0.0)	1 (0.25)	0 (0.0)	2 (0.50)	0 (0.0)	4

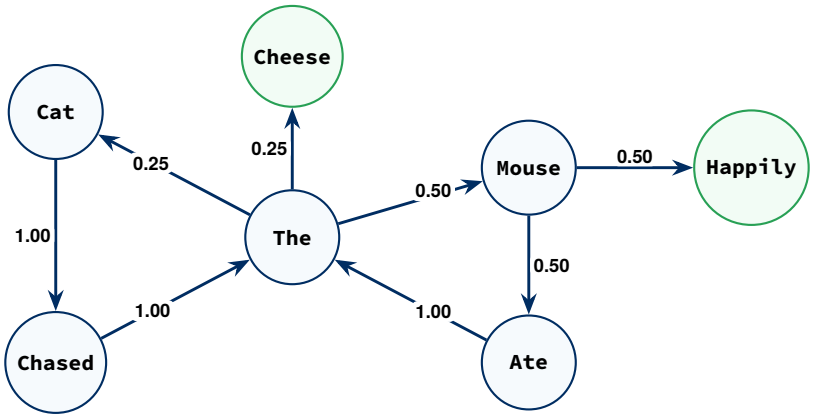


Figure 1.2: Markov state transition graph for the 7-token cat/mouse corpus. Directed edges indicate bigram transition probabilities $\mathbb{P}(w_t \mid w_{t-1})$. States `Cheese` and `Happily` (green) are terminal absorbing states with zero outgoing probability mass.

1.4 Autoregressive estimation and generation

With the transition matrix **M** constructed (Table 1.2) and its state graph mapped (Figure 1.2), the language model can execute two distinct operational tasks:

1. **Sequence Probability Estimation**: Assign a likelihood score to any proposed test sentence.
2. **Autoregressive Generation**: Incrementally sample tokens to produce novel synthetic text.

1.4.1 Worked sentence probability calculation

Consider evaluating the joint probability of the sentence:

"the cat chased the cheese"

Under the first-order Markov assumption, the joint probability factors as:

$$\mathbb{P}(\text{the, cat, chased, the, cheese}) = \mathbb{P}(\text{the})\cdot\mathbb{P}(\text{cat} \mid \text{the})\cdot\mathbb{P}(\text{chased} \mid \text{cat})\cdot\mathbb{P}(\text{the} \mid \text{chased})\cdot\mathbb{P}(\text{cheese} \mid \text{the}). \tag{1.9}$$

Assuming our generation begins deterministically with the initial token $\mathbb{P}(\text{the}) = 1.0$, we substitute the corresponding conditional probabilities from Table 1.2:

$$\begin{aligned}\mathbb{P}(\text{cat} \mid \text{the}) &= \frac{1}{4} = 0.25\\ \mathbb{P}(\text{chased} \mid \text{cat}) &= 1.0\\ \mathbb{P}(\text{the} \mid \text{chased}) &= 1.0\\ \mathbb{P}(\text{cheese} \mid \text{the}) &= \frac{1}{4} = 0.25\end{aligned}$$

Multiplying these values step by step:

$$\mathbb{P}(\text{the, cat, chased, the, cheese}) = 1.0 \times \frac{1}{4} \times 1 \times 1 \times \frac{1}{4} = \frac{1}{16} = 0.0625. \tag{1.10}$$

Despite the sequence *"the cat chased the cheese"* never occurring as an entire sentence in the training corpus, the bigram model assigns it a strictly positive probability of 6.25%. By decomposing text into local constituent pairs, n-gram models exhibit rudimentary combinatorial generalization.

1.4.2 5-step autoregressive generation trace

Now consider using the bigram model for open-ended text generation. Autoregressive sampling begins with a seed prefix, samples the next token from the resulting conditional probability distribution, appends the sampled token to the history, and repeats.

We trace the exact 5-step generation sequence recorded in the lecture slides, starting from the seed token *"the"*:

Step 1: Condition on prefix: history = ("the").

- Active row in transition matrix: Row "The".
- Candidate distribution: $\mathbb{P}(\text{cat} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{cheese} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{mouse} \mid \text{the}) = 0.50$.
- Random sample: **"mouse"**.
- Generated prefix: *"the mouse"*.

Step 2: Condition on prefix: history = (. . . , "mouse").

- Active row in transition matrix: Row "Mouse".
- Candidate distribution: $\mathbb{P}(\text{ate} \mid \text{mouse}) = 0.50$, $\mathbb{P}(\text{happily} \mid \text{mouse}) = 0.50$.
- Random sample: **"ate"**.
- Generated prefix: *"the mouse ate"*.

Step 3: Condition on prefix: history = (. . . , "ate").

- Active row in transition matrix: Row "Ate".
- Candidate distribution: $\mathbb{P}(\text{the} \mid \text{ate}) = 1.00$ (deterministic transition).
- Emitted token: **"the"**.
- Generated prefix: *"the mouse ate the"*.

Step 4: Condition on prefix: history = (. . . , "the").

- Active row in transition matrix: Row "The".
- Candidate distribution: $\mathbb{P}(\text{cat} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{cheese} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{mouse} \mid \text{the}) = 0.50$.
- Random sample: **"mouse"**.
- *Slide Note*: **:(** (A repetitive structural loop has emerged because the model cannot remember that it already emitted "mouse" two words ago).
- Generated prefix: *"the mouse ate the mouse"*.

Step 5: Condition on prefix: history = (. . . , "mouse").

- Active row in transition matrix: Row "Mouse".
- Candidate distribution: $\mathbb{P}(\text{ate} \mid \text{mouse}) = 0.50$, $\mathbb{P}(\text{happily} \mid \text{mouse}) = 0.50$.
- Random sample: **"happily"**.
- *Slide Note*: **(shocked face)** (A syntactically valid but bizarre, cannibalistic sentence is generated).
- Generated sequence: ***"The mouse ate the mouse happily"***.

Because the word *"happily"* has zero outgoing transitions in Table 1.2, the process reaches an absorbing terminal state and generation halts naturally.

Analysis of absorbing terminal states and infinite loops. During the sequence generation analysis, two fundamental edge cases emerge:

1. **Absorbing terminal states after sentence-ending tokens**: In Table 1.2, terminal sentence-ending words have a row count of zero ($C(\text{Happily}) = 0$), making the conditional probability division $0/0$ mathematically undefined. In standard production language modeling, this is formalized by appending an explicit end-of-sequence token $\langle /s \rangle$ (or [EOS]) to every training sentence, guaranteeing that every word possesses a non-zero outgoing transition to either another word or $\langle /s \rangle$.

2. **Infinite cyclic loops under greedy decoding**: Because bigram models condition exclusively on the single preceding token w_{t-1}, once the model transitions from *"ate"* to *"the"*, it retains zero memory that it already described the mouse eating cheese two steps earlier. If greedy decoding is used (arg max), the transition path becomes fully deterministic and falls into a stationary limit cycle: *"the* $\rightarrow$ *mouse* $\rightarrow$ *ate* $\rightarrow$ *the* $\rightarrow$ *mouse* $\rightarrow$ . . . *"*. Modern LLMs resolve this through long-context self-attention horizons, temperature-controlled stochastic sampling, and repetition penalties.

1.5 Fundamental limitations of statistical n-gram models: sparsity and zero-frequency collapse

While n-gram models provided the foundation for speech recognition, statistical machine translation, and spelling correction throughout the 1980s and 1990s, they suffer from fatal structural limitations that ultimately motivated the deep learning revolution.

1.5.1 Catastrophic data sparsity and parameter explosion

To condition on $n-1$ words, an n-gram model requires storing a discrete probability table of dimensionality $|\mathcal{V}|^n$. For realistic natural language and software applications:

- Vocabulary size: $|\mathcal{V}| \approx 50{,}000$ to $100{,}000$ unique words or code identifiers.
- Bigram table ($n = 2$): $|\mathcal{V}|^2 = 50{,}000^2 = 2.5 \times 10^9$ parameters ($\approx$ 10 GB at 32-bit floating point).
- Trigram table ($n = 3$): $|\mathcal{V}|^3 = 50{,}000^3 = 1.25 \times 10^{14}$ parameters (over 500 Terabytes).
- 4-gram table ($n = 4$): $|\mathcal{V}|^4 = 50{,}000^4 = 6.25 \times 10^{18}$ parameters (Exabyte scale).

Even if storage were free, the number of distinct 4-grams in all digitized human literature is dwarfed by 10^{18}. Consequently, the overwhelming majority of valid phrases never appear in training corpora.

1.5.2 The zero-frequency catastrophe: probability and perplexity collapse

The most lethal consequence of combinatorial sparsity is the *zero-frequency catastrophe*. Under Maximum Likelihood Estimation, any n-gram that never appeared verbatim in the training text receives count $C = 0$, yielding:

$$\mathbb{P}_{\text{MLE}}(w_t \mid w_{t-n+1}^{t-1}) = \frac{C(w_{t-n+1}^{t})}{C(w_{t-n+1}^{t-1})} = 0. \tag{1.11}$$

By the probabilistic chain rule (Equation (1.2)), evaluating a test sequence $W_{\text{test}} = (w_1, \ldots, w_N)$ requires taking the product across all constituent conditional transitions:

$$\mathbb{P}(W_{\text{test}}) = \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_{<t}) = \mathbb{P}(w_1) \cdots 0 \cdots \mathbb{P}(w_N \mid w_{<N}) = 0. \tag{1.12}$$

A single unobserved token transition collapses the estimated joint probability of an entire document, article, or codebase to absolute zero.

Furthermore, when evaluating the model via Empirical Cross-Entropy $\mathcal{H}(W_{\text{test}})$ or Perplexity $\text{PPL}(W_{\text{test}})$:

$$\mathcal{H}(W_{\text{test}}) = -\frac{1}{N} \sum_{t=1}^{N} \log_2 \mathbb{P}(w_t \mid w_{<t}) \longrightarrow \infty, \quad \text{PPL}(W_{\text{test}}) = 2^{\mathcal{H}(W_{\text{test}})} \longrightarrow \infty. \tag{1.13}$$

An unobserved test n-gram results in infinite perplexity ($\text{PPL} = \infty$), rendering the entire language model statistically useless for evaluation or comparison.

1.5.3 Finite context horizon limitations

An n-gram model is strictly blind to any token that occurred more than $n-1$ steps in the past:

$$\mathbb{P}(w_t \mid w_1, w_2, \ldots, w_{t-1}) \approx \mathbb{P}(w_t \mid w_{t-n+1}, \ldots, w_{t-1}). \tag{1.14}$$

Because the conditioning window is rigidly restricted to the preceding $n-1$ tokens, any semantic constraint, topic coherence, or syntactic agreement established earlier in the discourse is discarded. This finite context horizon causes severe memory amnesia across two critical domains:

Linguistic agreement breakdown in natural language. Consider long-range syntactic agreements where an arbitrary number of modifiers, prepositional phrases, or subordinate relative clauses separate the grammatical subject from its governing predicate:

*"The **engineers** who designed the complex distributed compiler infrastructure [...] **were** vs **was** awarded the patent."*

If $n = 4$ (a 4-gram model), the model predicts the verb by conditioning exclusively on the three preceding tokens:

$$\mathbb{P}(\text{verb} \mid \text{compiler}, \text{infrastructure}, [\ldots]).$$

Because the singular noun *"infrastructure"* is immediately adjacent to the verb position, the local 4-gram transition statistics strongly favor the singular verb *"was"*. The model has completely forgotten the true plural antecedent subject *"engineers"*, producing an ungrammatical agreement violation.

Structural and scoping breakdown in software engineering code. In programming languages, code validity requires strict, long-range determinism governed by context-free grammars and scope resolution rules:

1. **Lexical Scoping and Variable Lifecycles**: A variable declared in a function header:

   ```
   void process_batch(std::vector<TransactionRequest>& transactions) { ...}
   ```

   must be invoked using valid `std::vector` member methods (e.g., `.push_back()`, `.size()`) thirty lines later. An n-gram model sees only `transactions.` without access to the declaration, leading to syntactically invalid completions.

2. **Bracket and Block Matching**: Program code requires exact hierarchical pairing of opening and closing delimiters (`{ ...}`, `[ ...]`, `( ...)`) and control flow constructs (`try ...catch ...finally`). Because n-gram models lack internal state stacks, they frequently emit mismatched braces or premature block terminations.

1.5.4 Discrete orthogonal symbols and lack of semantic generalization

In classical statistical NLP, words are treated as discrete, orthogonal atomic symbols. If the model observes:

*"The **cat** chased the mouse happily"*

it gains zero statistical confidence regarding the identical sentence containing synonyms:

*"The **feline** chased the mouse happily"*

Because *"cat"* and *"feline"* are completely separate row indices in Table 1.2, observations of one do not inform the other. There is no concept of semantic distance, synonymy, or conceptual overlap.

The causal imperative. While long-range memory amnesia and semantic blindness demand continuous neural architectures (introduced in Chapters **??** and **??**), the immediate, fatal crisis in statistical language modeling is the **zero-frequency catastrophe** ($C = 0 \implies P = 0 \implies \text{PPL} = \infty$). Before we can even evaluate an n-gram model, we must establish rigorous mathematical algorithms that steal probability mass from frequent events and reallocate it to unobserved events: this is the domain of **statistical smoothing**.

1.6 Statistical smoothing and discounting algorithms

To prevent probability collapse ($P = 0$) and infinite perplexity ($\text{PPL} = \infty$) on unseen test sequences, statistical language modeling relies on *smoothing* (or discounting). The governing principle of smoothing is conservative probability reallocation: we shave off a fraction of probability mass from observed events and redistribute it across unseen events, ensuring that every valid sequence receives a strictly positive probability:

$$\forall w \in \mathcal{V}, \quad \mathbb{P}(w \mid \text{context}) > 0.$$

1.6.1 Laplace add-one and add-α smoothing

The simplest smoothing formulation, originated by Pierre-Simon Laplace, adds a uniform pseudo-count $\alpha > 0$ to every possible transition.

Mathematical formulation. For an unconditioned unigram model, Laplace add-α smoothing assigns:

$$\mathbb{P}_{\text{Laplace}}(w) = \frac{C(w) + \alpha}{N + \alpha|\mathcal{V}|}, \tag{1.15}$$

where $N = \sum_{w \in \mathcal{V}} C(w)$ is the total token count. For a bigram model conditioned on w_{t-1}, the estimator modifies Equation (1.8) as:

$$\mathbb{P}_{\text{Laplace}}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t) + \alpha}{C(w_{t-1}) + \alpha|\mathcal{V}|}. \tag{1.16}$$

When $\alpha = 1$, the technique is known as *Laplace smoothing* (or Add-1); when $0 < \alpha < 1$, it is termed *Lidstone smoothing*.

Proof of proper probability distribution. We verify that Equation (1.16) satisfies the Kolmogorov axioms by summing over the entire vocabulary $\mathcal{V}$:

$$\begin{aligned}
\sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{Laplace}}(w_t \mid w_{t-1}) &= \sum_{w_t \in \mathcal{V}} \frac{C(w_{t-1}, w_t) + \alpha}{C(w_{t-1}) + \alpha|\mathcal{V}|} \\
&= \frac{\sum_{w_t \in \mathcal{V}} C(w_{t-1}, w_t) + \sum_{w_t \in \mathcal{V}} \alpha}{C(w_{t-1}) + \alpha|\mathcal{V}|} \\
&= \frac{C(w_{t-1}) + \alpha|\mathcal{V}|}{C(w_{t-1}) + \alpha|\mathcal{V}|} = 1.
\end{aligned} \tag{1.17}$$

Why Add-1 fails in practice. Although mathematically clean, Add-1 smoothing performs terribly in natural language processing. In realistic vocabularies where $|\mathcal{V}| = 50{,}000$, if word w_{t-1} occurred $C(w_{t-1}) = 100$ times, adding $\alpha|\mathcal{V}| = 50{,}000$ inflates the denominator from 100 to 50,100. A frequent observed transition with $C(w_{t-1}, w_t) = 50$ is diluted from an empirical probability of 0.50 to:

$$\frac{50 + 1}{100 + 50{,}000} = \frac{51}{50{,}100} \approx 0.00102.$$

Add-1 smoothing steals more than 99.8% of the probability mass from observed events and reallocates it uniformly to unobserved transitions. Even Lidstone smoothing ($\alpha = 0.05$) assigns the same probability to all unseen words, failing to recognize that some unseen words (e.g., common nouns) are inherently more probable than rare technical jargon.

1.6.2 Good-Turing discounting

Originally formulated in [**?**] during cryptographic research at Bletchley Park, *Good-Turing discounting* derives the probability of events seen r times directly from the count of events seen $r + 1$ times.

Frequency of frequencies. Let $r = C(w_{t-n+1}^{t}) \geq 0$ denote the empirical frequency of an n-gram. We define the *frequency of frequencies* N_r as the total number of distinct n-grams that appeared exactly r times in the training corpus:

$$N_r = |\{x : C(x) = r\}|. \tag{1.18}$$

The total number of tokens in the corpus is $N = \sum_{r=1}^{\infty} rN_r$, and N_0 represents the enormous number of valid n-grams that never appeared ($r = 0$).

Adjusted count derivation. Good-Turing replaces each raw count r with an effective discounted count r^*:

$$r^* = (r + 1)\frac{N_{r+1}}{N_r}. \tag{1.19}$$

For any observed n-gram with count $r > 0$, its Good-Turing smoothed probability is:

$$\mathbb{P}_{\text{GT}}(w_t \mid w_{t-n+1}^{t-1}) = \frac{r^*}{N} = \frac{(r + 1)N_{r+1}}{N \cdot N_r}. \tag{1.20}$$

The total probability mass reserved for all unobserved n-grams ($r = 0$) is determined by the proportion of *singletons* (N_1, words seen exactly once):

$$P_0 = \frac{N_1}{N}. \tag{1.21}$$

Because there are N_0 unobserved n-grams, each individual unseen n-gram receives probability:

$$\mathbb{P}_{\text{GT}}(\text{unseen}) = \frac{P_0}{N_0} = \frac{N_1}{N \cdot N_0}. \tag{1.22}$$

Smoothing N_r for large r. For large values of r, empirical counts become sparse (N_r fluctuates wildly or drops to zero, making N_{r+1}/N_r undefined). In production systems, *Simple Good-Turing* [**?**] replaces raw N_r values with smoothed estimates derived from a linear regression fit in log space ($\log N_r = a + b \log r$) for $r > k$.

1.6.3 Katz backoff with α normalization

The recursive framework introduced in [**?**] extended Good-Turing discounting through a hierarchical backoff mechanism: if an n-gram has been observed, use its discounted probability; if it has never been observed, back off to the $(n - 1)$-gram distribution, scaled by a normalizing factor.

Mathematical formulation. For a bigram model, the Katz backoff estimator is:

$$\mathbb{P}_{\text{Katz}}(w_t \mid w_{t-1}) = \begin{cases} \mathbb{P}^*(w_t \mid w_{t-1}) & \text{if } C(w_{t-1}, w_t) > 0, \\ \alpha(w_{t-1})\mathbb{P}_{\text{Katz}}(w_t) & \text{if } C(w_{t-1}, w_t) = 0, \end{cases} \tag{1.23}$$

where $\mathbb{P}^*(w_t \mid w_{t-1}) = \frac{C^*(w_{t-1}, w_t)}{C(w_{t-1})}$ represents the discounted probability calculated using Good-Turing adjusted counts for low frequencies ($r \leq k$, typically $k = 5$) and MLE for high counts ($r > k$):

$$C^*(w_{t-1}, w_t) = \frac{\frac{r^*}{r} - \frac{(k+1)N_{k+1}}{N_1}}{1 - \frac{(k+1)N_{k+1}}{N_1}} \cdot r \quad \text{for } 1 \leq r \leq k. \tag{1.24}$$

Derivation of the normalizing backoff weight $\alpha(w_{t-1})$. The leftover probability mass shaved off from observed bigrams is:

$$\beta(w_{t-1}) = 1 - \sum_{w_t : C(w_{t-1}, w_t) > 0} \mathbb{P}^*(w_t \mid w_{t-1}). \tag{1.25}$$

To ensure that $\sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{Katz}}(w_t \mid w_{t-1}) = 1$, the backoff weight $\alpha(w_{t-1})$ scales the lower-order unigram distribution over unobserved words:

$$\alpha(w_{t-1}) = \frac{\beta(w_{t-1})}{\sum_{w_t : C(w_{t-1}, w_t) = 0} \mathbb{P}_{\text{Katz}}(w_t)}. \tag{1.26}$$

Summing across the entire vocabulary confirms proper normalization:

$$\begin{aligned} \sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{Katz}}(w_t \mid w_{t-1}) &= \sum_{C>0} \mathbb{P}^*(w_t \mid w_{t-1}) + \sum_{C=0} \alpha(w_{t-1})\mathbb{P}_{\text{Katz}}(w_t) \\ &= \big(1 - \beta(w_{t-1})\big) + \alpha(w_{t-1}) \sum_{C=0} \mathbb{P}_{\text{Katz}}(w_t) \\ &= \big(1 - \beta(w_{t-1})\big) + \beta(w_{t-1}) = 1. \end{aligned} \tag{1.27}$$

1.6.4 Kneser-Ney smoothing and continuation probabilities

The pinnacle of statistical language modeling is *Kneser-Ney smoothing* [**?**]. Kneser-Ney introduces two transformative innovations: absolute discounting and the continuation probability.

Absolute discounting. Rather than scaling counts proportionally, empirical analysis of corpus frequencies demonstrates that subtracting a fixed constant $d \in (0, 1)$ (typically $d \approx 0.75$) from all positive counts accurately models the true discounted frequency:

$$C_{\text{discounted}} = \max\left(C(w_{t-n+1}^t) - d,\, 0\right).$$

The continuation probability failure mode of standard backoff. Consider predicting the missing word in the sentence:

"I can't see without my reading _____"

The word *"glasses"* should receive a high conditional probability. Now consider the word *"Francisco"*. In any standard text corpus, *"Francisco"* has a high unigram frequency because the proper noun *"San Francisco"* appears frequently. Under Katz backoff, because the bigram *"reading Francisco"* has zero count, the model backs off to the unigram frequency of *"Francisco"*, assigning it a dangerously high probability of completing the sentence!

The breakthrough insight in [**?**] recognized that the unigram backoff distribution should not reflect *how often* a word appears overall, but *how versatile* the word is across diverse preceding contexts.

Definition 1.4: *Continuation probability*

The *continuation probability* $\mathbb{P}_{\text{cont}}(w_t)$ is the ratio of the number of unique preceding word types that precede w_t to the total number of unique bigram types in the vocabulary:

$$\mathbb{P}_{\text{cont}}(w_t) = \frac{|\{w_{t-1} : C(w_{t-1}, w_t) > 0\}|}{\sum_{w'} |\{w_{t-1} : C(w_{t-1}, w') > 0\}|} = \frac{|\{w_{t-1} : C(w_{t-1}, w_t) > 0\}|}{|\{(w_{t-1}, w_t) : C(w_{t-1}, w_t) > 0\}|}. \tag{1.28}$$

Because *"Francisco"* almost exclusively appears preceded by the single word *"San"*, $|\{w_{t-1} : C(w_{t-1}, \text{Francisco}) > 0\}| \approx 1$, resulting in a tiny continuation probability. In contrast, words like *"glasses"* or *"book"* appear in dozens of distinct contexts, yielding high continuation probabilities.

Interpolated Kneser-Ney formulation. Combining absolute discounting with continuation probabilities yields the *Interpolated Kneser-Ney* estimator:

$$\mathbb{P}_{\text{KN}}(w_t \mid w_{t-n+1}^{t-1}) = \frac{\max\left(C(w_{t-n+1}^{t}) - d,\, 0\right)}{C(w_{t-n+1}^{t-1})} + \lambda(w_{t-n+1}^{t-1})\mathbb{P}_{\text{KN}}(w_t \mid w_{t-n+2}^{t-1}), \tag{1.29}$$

where $\lambda(w_{t-n+1}^{t-1})$ is an explicit normalizing interpolation weight:

$$\lambda(w_{t-n+1}^{t-1}) = \frac{d}{C(w_{t-n+1}^{t-1})} \cdot \left|\{w : C(w_{t-n+1}^{t-1}, w) > 0\}\right|. \tag{1.30}$$

For a bigram model ($n = 2$), the base recursion terminates at the unigram continuation probability:

$$\mathbb{P}_{\text{KN}}(w_t \mid w_{t-1}) = \frac{\max\left(C(w_{t-1}, w_t) - d,\, 0\right)}{C(w_{t-1})} + \frac{d}{C(w_{t-1})} \cdot |\{w : C(w_{t-1}, w) > 0\}| \cdot \mathbb{P}_{\text{cont}}(w_t). \tag{1.31}$$

Proof of proper normalization. Summing Equation (1.31) over all $w_t \in \mathcal{V}$:

$$\begin{aligned}
\sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{KN}}(w_t \mid w_{t-1}) &= \sum_{w_t : C>0} \frac{C(w_{t-1}, w_t) - d}{C(w_{t-1})} + \lambda(w_{t-1}) \sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{cont}}(w_t) \\
&= \frac{\sum_{C>0} C(w_{t-1}, w_t) - d \cdot |\{w : C(w_{t-1}, w) > 0\}|}{C(w_{t-1})} + \lambda(w_{t-1}) \cdot 1 \\
&= \frac{C(w_{t-1}) - d|\{w : C(w_{t-1}, w) > 0\}|}{C(w_{t-1})} + \frac{d|\{w : C(w_{t-1}, w) > 0\}|}{C(w_{t-1})} = 1.
\end{aligned} \tag{1.32}$$

Interpolated Kneser-Ney consistently outperforms all other statistical smoothing techniques, serving as the industry standard until the advent of deep learning.

The causal imperative. Now that we have established competing smoothing algorithms (Laplace, Good-Turing, Katz, Kneser-Ney), each reallocating probability mass under different mathematical assumptions, we face a crucial practical question: *How do we quantitatively benchmark and compare these models on held-out evaluation text without running costly downstream applications?* This brings us directly to formal evaluation metrics.

1.7 Evaluation metrics: cross-entropy and perplexity

To benchmark and compare different language models quantitatively without executing costly downstream applications, natural language processing relies on intrinsic, information-theoretic metrics: *Cross-Entropy* and *Perplexity* [**?** **?**].

1.7.1 Information-theoretic foundations: entropy and KL divergence

Let $\mathcal{X}$ denote a discrete sample space (the vocabulary $\mathcal{V}$), and let $P(x)$ represent the true, underlying, but inaccessible probability distribution of human language.

Definition 1.5: *Shannon entropy*

The *Shannon entropy* $H(P)$ measures the fundamental, irreducible uncertainty or average information content inherent in distribution P [**?**]:

$$H(P) = -\sum_{x \in \mathcal{X}} P(x) \log_2 P(x) = \mathop{\mathbb{E}}_{x \sim P} \left[\log_2 \frac{1}{P(x)} \right]. \tag{1.33}$$

By Shannon's source coding theorem, $H(P)$ represents the absolute theoretical lower bound on the expected number of bits required to encode a symbol emitted by P without loss.

When constructing a language model, we approximate the true distribution P with an estimated parameterized distribution Q_θ. To quantify the statistical discrepancy between the true distribution P and our approximating model Q_θ, we employ the *Kullback-Leibler (KL) divergence*.

Definition 1.6: *Kullback-Leibler (KL) divergence*

The *KL divergence* (or relative entropy) from model distribution Q to target distribution P is:

$$D_{\mathrm{KL}}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log_2 \left(\frac{P(x)}{Q(x)} \right) = \mathop{\mathbb{E}}_{x \sim P} \left[\log_2 \frac{P(x)}{Q(x)} \right]. \tag{1.34}$$

Theorem 1.1: *Gibbs' inequality*

For any two discrete probability distributions P and Q, the KL divergence is non-negative:

$$D_{\mathrm{KL}}(P \parallel Q) \geq 0, \tag{1.35}$$

with equality $D_{\mathrm{KL}}(P \parallel Q) = 0$ if and only if $P(x) = Q(x)$ for all $x \in \mathcal{X}$.

Proof. Applying the natural logarithm and utilizing the standard inequality $\ln(u) \leq u - 1$ for all $u > 0$ (with equality if and only if $u = 1$):

$$\begin{aligned} -D_{\mathrm{KL}}(P \parallel Q) &= \sum_{x \in \mathcal{X}} P(x) \log_2 \left(\frac{Q(x)}{P(x)} \right) = \frac{1}{\ln 2} \sum_{x \in \mathcal{X}} P(x) \ln \left(\frac{Q(x)}{P(x)} \right) \\ &\leq \frac{1}{\ln 2} \sum_{x \in \mathcal{X}} P(x) \left(\frac{Q(x)}{P(x)} - 1 \right) \\ &= \frac{1}{\ln 2} \left(\sum_{x \in \mathcal{X}} Q(x) - \sum_{x \in \mathcal{X}} P(x) \right) = \frac{1}{\ln 2} (1 - 1) = 0. \end{aligned}$$

Multiplying both sides by -1 reverses the inequality, yielding $D_{\mathrm{KL}}(P \parallel Q) \geq 0$. ■

1.7.2 Cross-entropy decomposition

Expanding the logarithm in the definition of KL divergence demonstrates the fundamental relationship connecting Entropy, KL Divergence, and Cross-Entropy:

$$\begin{aligned} D_{\mathrm{KL}}(P \parallel Q) &= \sum_{x \in \mathcal{X}} P(x) \log_2 P(x) - \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) \\ &= -H(P) + \left(- \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) \right). \end{aligned} \tag{1.36}$$

Rearranging terms defines the *Cross-Entropy* $H(P, Q)$:

$$H(P, Q) \triangleq - \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) = H(P) + D_{\mathrm{KL}}(P \parallel Q). \tag{1.37}$$

Why optimizing cross-entropy optimizes KL divergence. Equation (1.37) reveals the theoretical justification for cross-entropy training:

- The true language distribution P is determined by human linguistic behavior; therefore, the true entropy $H(P)$ is a fixed, immutable physical constant.
- When we minimize cross-entropy with respect to model parameters θ:

$$\arg\min_{\theta} H(P, Q_\theta) = \arg\min_{\theta} \left(H(P) + D_{\mathrm{KL}}(P \parallel Q_\theta) \right) = \arg\min_{\theta} D_{\mathrm{KL}}(P \parallel Q_\theta). \tag{1.38}$$

Minimizing cross-entropy is mathematically identical to driving the KL divergence between model Q_θ and the true language distribution P to zero.

1.7.3 Empirical sequence cross-entropy

Because we do not have closed-form access to the true distribution $P(x)$, we cannot compute the expectation $\mathbb{E}_{x \sim P}[\cdot]$ directly. Instead, by the Shannon-McMillan-Breiman theorem (the ergodic theorem for stationary information sources), the expected value is estimated empirically by sampling a long test sequence $W = (w_1, w_2, \ldots, w_T) \sim P$:

$$\mathcal{H}(W) = -\frac{1}{T} \log_2 Q(w_1, w_2, \ldots, w_T) = -\frac{1}{T} \sum_{t=1}^{T} \log_2 Q(w_t \mid w_{<t}). \tag{1.39}$$

Empirical cross-entropy measures the average number of bits required to compress each token of test text using our model Q.

1.7.4 Perplexity (PPL)

While cross-entropy evaluates performance on a logarithmic scale, researchers and practitioners overwhelmingly report *Perplexity* (PPL), defined as the exponentiated cross-entropy loss:

$$\mathrm{PPL}(W) = 2^{\mathcal{H}(W)} = 2^{-\frac{1}{T}\sum_{t=1}^{T}\log_2 Q(w_t|w_{<t})} \tag{1.40}$$

$$= \left(\prod_{t=1}^{T}\frac{1}{Q(w_t \mid w_{<t})}\right)^{\frac{1}{T}}. \tag{1.41}$$

Perplexity represents the reciprocal geometric mean of the token probabilities assigned by the model across the test corpus.

1.7.5 Rigorous proof: perplexity as the effective branching factor

The fundamental intuition taught across NLP literature is that perplexity quantifies the *effective branching factor* of a language model. We now formalize this property through a rigorous proof.

Theorem 1.2: *Perplexity of a uniform K-ary distribution*

Let a language model Q predict next tokens such that at each timestep t, the probability mass is distributed uniformly across exactly K equally likely candidate tokens ($Q(w_t \mid w_{<t}) = \frac{1}{K}$ for each candidate). Then the perplexity on any generated sequence W of length T is identically K:

$$\mathrm{PPL}(W) = K. \tag{1.42}$$

Proof. Substituting the uniform prediction probability $Q(w_t \mid w_{<t}) = \frac{1}{K}$ into the empirical cross-entropy formula:

$$\mathcal{H}(W) = -\frac{1}{T}\sum_{t=1}^{T}\log_2\left(\frac{1}{K}\right) = -\frac{1}{T}\sum_{t=1}^{T}\left(-\log_2 K\right)$$
$$= \frac{1}{T}\left(T\log_2 K\right) = \log_2 K.$$

Exponentiating the cross-entropy:

$$\mathrm{PPL}(W) = 2^{\mathcal{H}(W)} = 2^{\log_2 K} = K. \quad \blacksquare$$

Physical interpretation. When a model evaluates a text corpus and achieves PPL = 42, it indicates that, on average, the model's predictive uncertainty at each token is mathematically equivalent to choosing uniformly at random from a shortlist of 42 equally plausible words:

- A uniform random guesser across vocabulary $\mathcal{V}$ achieves PPL = $|\mathcal{V}|$ (e.g., PPL = 50,000).
- An optimal oracle that predicts every token with probability 1.0 achieves the theoretical lower bound PPL = 1.0.
- State-of-the-art LLMs on natural language benchmarks typically achieve perplexities between 8 and 15.

1.7.6 Base-2 vs. base-e logarithmic conversions and perplexity computation

A critical mathematical nuance arises when connecting information-theoretic formulations (which traditionally employ base-2 logarithms) with deep learning frameworks such as PyTorch (which exclusively compute natural base-e cross-entropy loss):

Property	Information Theory (Bits)	Deep Learning / PyTorch (Nats)
Loss Formula	$\mathcal{H}_2 = -\frac{1}{T}\sum \log_2 Q(w_t \mid w_{<t})$	$\mathcal{L}_{\text{CE}} = -\frac{1}{T}\sum \ln Q(w_t \mid w_{<t})$
Unit	Bits per token	Nats per token
Perplexity Formula	$\text{PPL} = 2^{\mathcal{H}_2}$	$\text{PPL} = \exp(\mathcal{L}_{\text{CE}}) = e^{\mathcal{L}_{\text{CE}}}$
Conversion Identity	$\mathcal{H}_2 = \frac{\mathcal{L}_{\text{CE}}}{\ln 2} \approx \frac{\mathcal{L}_{\text{CE}}}{0.693147}$,	$\mathcal{L}_{\text{CE}} = \mathcal{H}_2 \cdot \ln 2 \approx 0.693147 \cdot \mathcal{H}_2$

Worked numerical calculation. Suppose a model trained on code completion reports an evaluation cross-entropy loss of $\mathcal{L}_{\text{CE}} = 2.7726$ nats:

1. **Direct Base-e Perplexity**:
$$\text{PPL} = e^{2.7726} = \exp(2.7726) \approx 16.00.$$
2. **Conversion to Information Bits**:
$$\mathcal{H}_2 = \frac{2.7726}{\ln 2} = \frac{2.7726}{0.693147} = 4.00 \text{ bits/token}.$$
3. **Base-2 Perplexity Verification**:
$$\text{PPL} = 2^{\mathcal{H}_2} = 2^{4.00} = 16.00.$$

Notice that evaluating $2^{\mathcal{L}_{\text{CE}}} = 2^{2.7726} \approx 6.83$ would severely underestimate the true branching factor by more than 57%. One must always ensure consistent logarithmic bases when deriving perplexity from loss.

1.7.7 The glass ceiling of smoothed n-gram models

While Kneser-Ney smoothing dramatically reduces test perplexity by eliminating zero-frequency collapses, smoothed n-gram models remain fundamentally handicapped:

1. **Horizon Truncation**: Conditioning remains locked to $n-1$ tokens; increasing $n \geq 5$ yields negligible gains because higher-order frequencies are almost entirely unobserved, causing the model to constantly back off to lower orders.
2. **Zero Parameter Sharing**: Words remain orthogonal indices. The model cannot generalize that *"The programmer wrote a test"* informs *"The developer authored a test"*.
3. **Combinatorial Storage**: Counting tables for large corpora require gigabytes to terabytes of RAM.

These structural limits motivated researchers to explore new paradigms that replace discrete counting tables with continuous, parameterized functions.

1.8 Historical evolution of language models

Understanding modern large language models requires appreciating the 75-year evolutionary trajectory that progressively dismantled the mathematical and computational barriers of discrete symbol counting. Figure 1.3 synthesizes the macro-historical progression of language modeling from Claude Shannon's foundational 1948 information theory to modern autonomous software engineering systems.

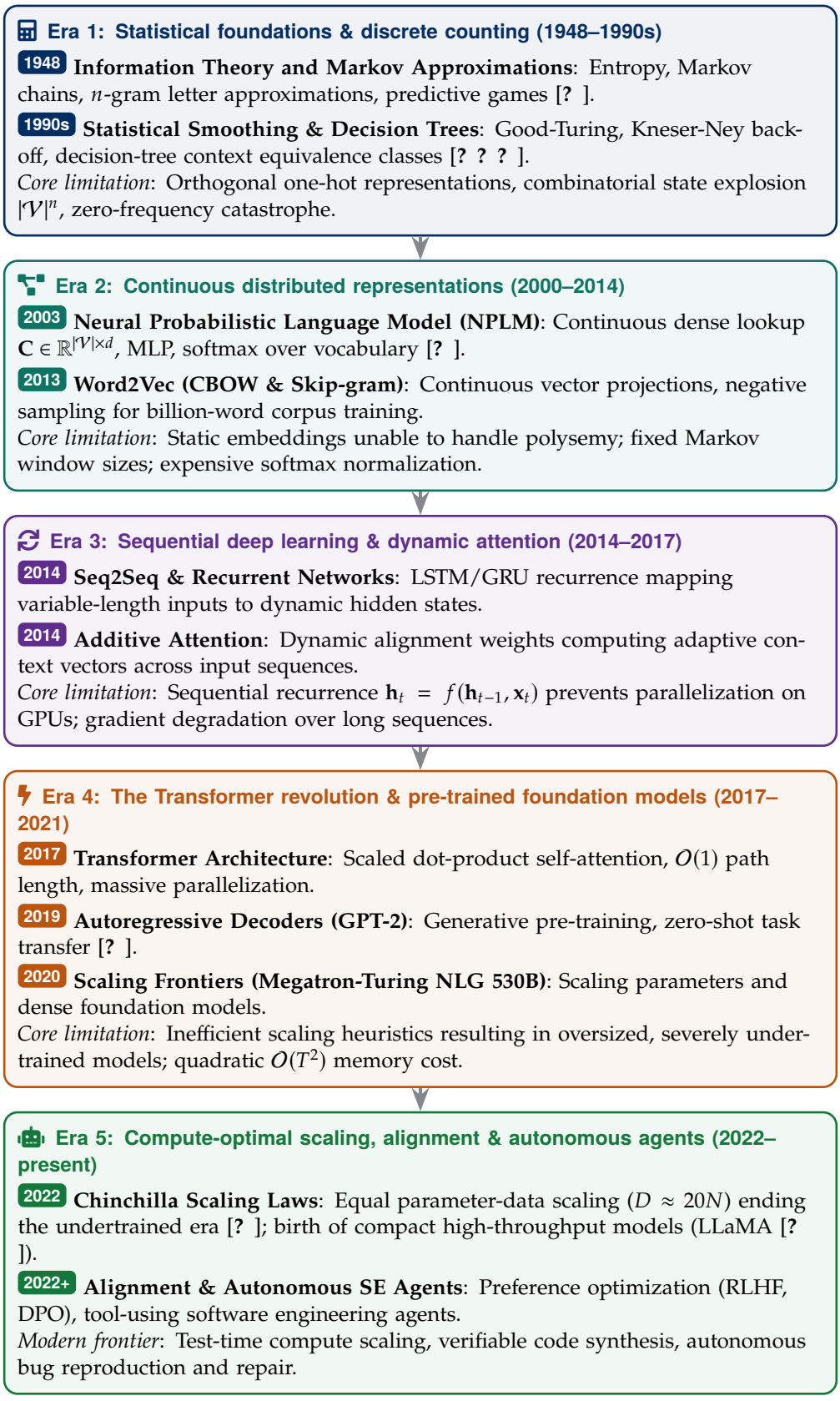


Figure 1.3: Chronological timeline of language model evolution from 1948 to the present.

1.8.1 1948–1980s: Shannon's information theory and Markov n-grams

The conceptual origin of language modeling stems from the landmark 1948 paper *"A Mathematical Theory of Communication"* [**?**]. In that work, written human language was treated as a discrete stochastic process generated by an underlying stationary Markov source over a finite alphabet.

Shannon introduced the *Shannon game*—a pedagogical thought experiment where an observer attempts to predict the next letter in an unknown sequence of text given the preceding letters. Shannon demonstrated that as the conditioning context length increases from zero-order (independent uniform character selection) to unigram (individual character frequencies), bigram (pairs), and trigram (triplets) models, the generated text rapidly converges toward natural English orthography and syntax.

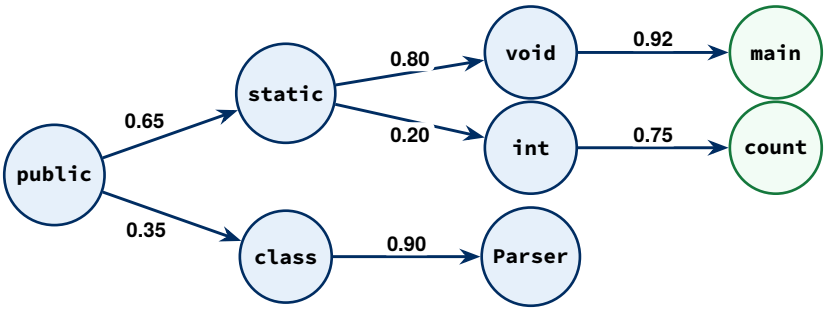


Figure 1.4: Markov chain state transition graph for software tokens.

In this framework, the conditional probability of token w_t given context $w_1, \ldots, w_{t-1}$ is simplified via an order-$(n-1)$ Markov assumption, as illustrated in Figure 1.4 for code syntax.

1.8.2 1990s: statistical decision trees and context partitioning

Throughout the 1990s, as speech recognition and machine translation systems scaled, researchers recognized that flat n-gram tables could not condition on long histories without exponential parameter explosion.

To overcome this limitation in the course progression, *decision-tree language models* were introduced in [**?**] (as examined on Slide 19 of the course lecture deck). Rather than conditioning probability on a rigid, exact sequence of $n-1$ tokens, a binary decision tree asks a sequence of syntactic and lexical questions about the historical context $\mathcal{H} = (w_{t-1}, w_{t-2}, \ldots)$.

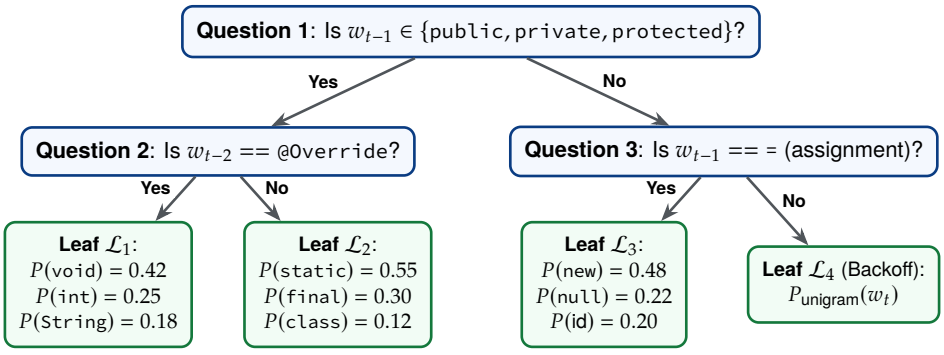


Figure 1.5: Decision tree context partitioning for conditional language modeling [**?**].

As illustrated in Figure 1.5, context histories that share identical predictive characteristics are clustered into the same leaf node $\mathcal{L}_k$, yielding conditional distributions $\mathbb{P}(w_t \mid \mathcal{L}_k)$. While this reduced the number of free parameters compared to full tables, the model remained fundamentally bound to discrete categorical splitting without true semantic interpolation.

1.8.3 2000–2003: The neural probabilistic language model

In the landmark 2003 paper *"A Neural Probabilistic Language Model"* [**?**], natural language processing was revolutionized by introducing continuous distributed representations to defeat the curse of dimensionality (as highlighted on Slide 20 of the course lectures).

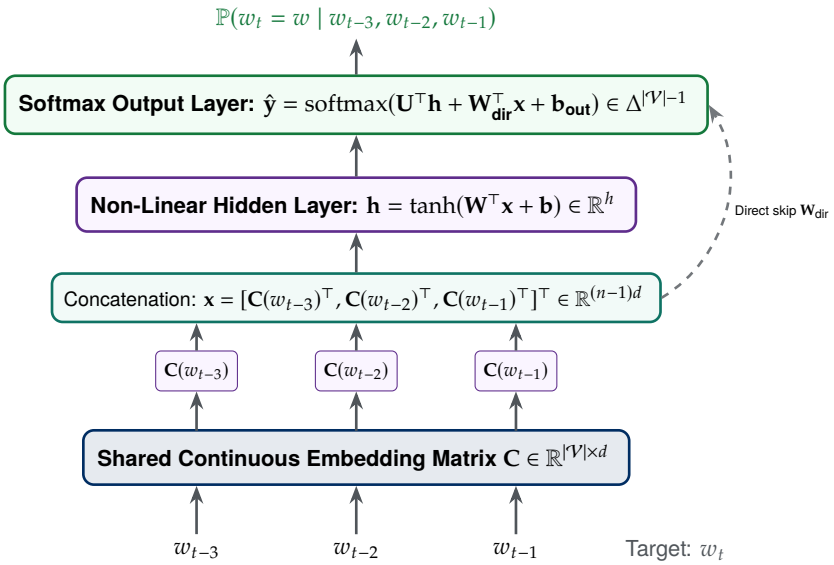


Figure 1.6: The Neural Probabilistic Language Model (NPLM) architecture [**?**]. Discrete context words are mapped into continuous dense feature vectors via shared matrix $\mathbf{C} \in \mathbb{R}^{|\mathcal{V}| \times d}$, concatenated, and processed through a non-linear hidden layer to predict next-token probabilities via softmax.

The neural probabilistic formulation in [**?**] replaced discrete count tables with a continuous parameterized architecture (Figure 1.6):

1. **Continuous Embedding Projection**: Each word $w \in \mathcal{V}$ is associated with a real-valued distributed feature vector $\mathbf{c}_w \in \mathbb{R}^d$ ($d \ll |\mathcal{V}|$, e.g., $d = 100$). The vocabulary lookup matrix $\mathbf{C} \in \mathbb{R}^{|\mathcal{V}| \times d}$ is learned jointly via backpropagation.

2. **Semantic Smoothing via Continuous Space**: If *"cat"* and *"feline"* have similar embedding vectors ($\mathbf{c}_{\text{cat}} \approx \mathbf{c}_{\text{feline}}$), observing *"the cat chased the mouse"* automatically increases the model's confidence in *"the feline chased the mouse"*. Continuous geometry solves the orthogonal symbol limitation.

3. **Softmax Output Normalization**: The network projects hidden activations to $|\mathcal{V}|$ unnormalized logits and applies the softmax function to guarantee a valid probability distribution:

$$\mathbb{P}(w_t = i \mid w_{<t}) = \frac{\exp(z_i)}{\sum_{j=1}^{|\mathcal{V}|} \exp(z_j)}. \tag{1.43}$$

1.8.4 2018–2019: Generative pre-training and GPT-2

While continuous representations were established in [**?**], models throughout the 2000s and 2010s remained restricted in scale and context length. In 2018–2019, the GPT-2 architecture was introduced in [**?**] (*"Language Models are Unsupervised Multitask Learners"*, examined on Slide 21 of the course slides).

GPT-2 demonstrated that massive, autoregressive decoder-only models trained on hundreds of gigabytes of uncurated internet text (WebText) could perform diverse natural language

and software tasks without task-specific supervised training. By predicting next tokens over context windows of 1,024 tokens, the model exhibited emergent zero-shot transfer across reading comprehension, machine translation, and Python code generation.

1.8.5 2020–2022: Megatron-Turing NLG 530B and compute-optimal scaling

Following GPT-2 and GPT-3, the deep learning industry initiated an aggressive scaling arms race, culminating in monolithic models such as the Megatron-Turing NLG 530B (Slide 22). However, early scaling heuristics [**?**] recommended scaling model parameters faster than training data. Consequently, models like Megatron-Turing NLG 530B and GPT-3 (175B parameters trained on 300B tokens) were severely **undertrained** relative to their parameter capacity.

In 2022, the landmark study *"Training Compute-Optimal Large Language Models"* [**?**] established the **Chinchilla** scaling laws:

$$D \approx 20 \times N, \tag{1.44}$$

where N is parameter count and D is training tokens. For compute optimality, parameter count and training tokens must scale in equal proportion. This insight prompted the modern era of compact, heavily trained open-weight models (e.g., LLaMA [**?**] trained on trillions of tokens), dramatically reducing deployment and inference overhead in software engineering environments.

1.8.6 From discrete counting tables to continuous parametric learners: bridge to deep learning

Our exploration of statistical language models in this chapter has revealed a fundamental evolutionary trajectory:

- **The Discrete Counting Limit**: Exact probability factorization via the chain rule (Equation (1.2)) is theoretically exact, but empirical n-gram counting tables suffer from exponential state-space explosion (V^n), context horizon amnesia ($n-1$), and orthogonal semantic blindness.
- **The Smoothing Ceiling**: While smoothing algorithms (Good-Turing, Katz, Kneser-Ney) rescue models from zero-frequency collapse ($P = 0 \implies \text{PPL} = \infty$), they merely redistribute probability mass across discrete table bins. They cannot learn high-dimensional feature interactions or share statistical strength between synonymous phrases.

To transcend the discrete counting paradigm, natural language processing abandoned frequency tables in favor of **continuous, parameterized neural networks**:

$$f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{W}^\top \mathbf{x} + \mathbf{b}.$$

Instead of counting co-occurrences in text, we parameterize language predictors as continuous functions whose weights $\boldsymbol{\theta}$ are optimized iteratively through gradient descent and backpropagation.

This transition requires establishing the mathematical and computational foundations of deep learning: How does an artificial neuron compute? Why is stacking linear layers without non-linear activations mathematically futile? What allows multi-layer perceptrons to approximate arbitrary complex functions? And how does backpropagation automatically compute gradients across computational graphs?

We address each of these foundational questions in Chapter **??**: *Deep Learning Foundations for NLP*.